

System Design

Monolithos e
Microserviços



A diagram consisting of a dashed rectangular box and a solid rectangular box. The dashed box is positioned above the solid box. A dashed curved arrow points from the top-right corner of the dashed box to the top-right corner of the solid box. A solid vertical arrow points from the bottom-right corner of the solid box upwards to the bottom-right corner of the dashed box. A dashed curved arrow points from the bottom-left corner of the solid box to the bottom-left corner of the dashed box.

\$ whoami

Matheus Fidelis

Engenheiro de \$RANDOM

@fidelissauro

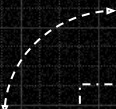
<https://fidelissauro.dev>

<https://linktr.ee/fidelissauro>



OBJETIVOS

- ✓ Análise Pragmática e Técnica
- ✓ Arquitetura Monolítica
- ✓ Arquitetura de Microserviços
- ✓ Tradeoffs
- ✓ Introdução a Design de Domínio
- ✓ Lei de Conway



1

Arquitetura Monolítica



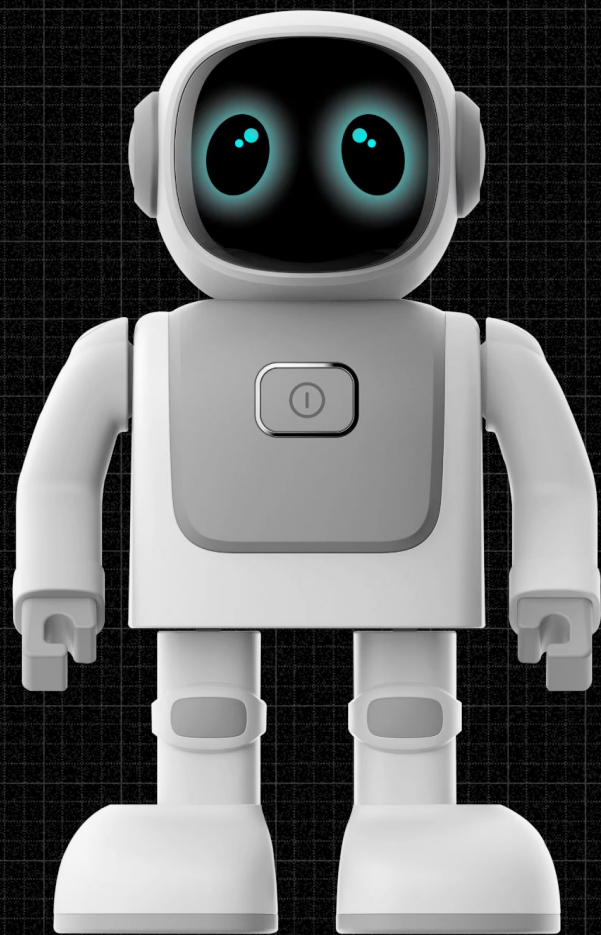
Definições e
Conceitos





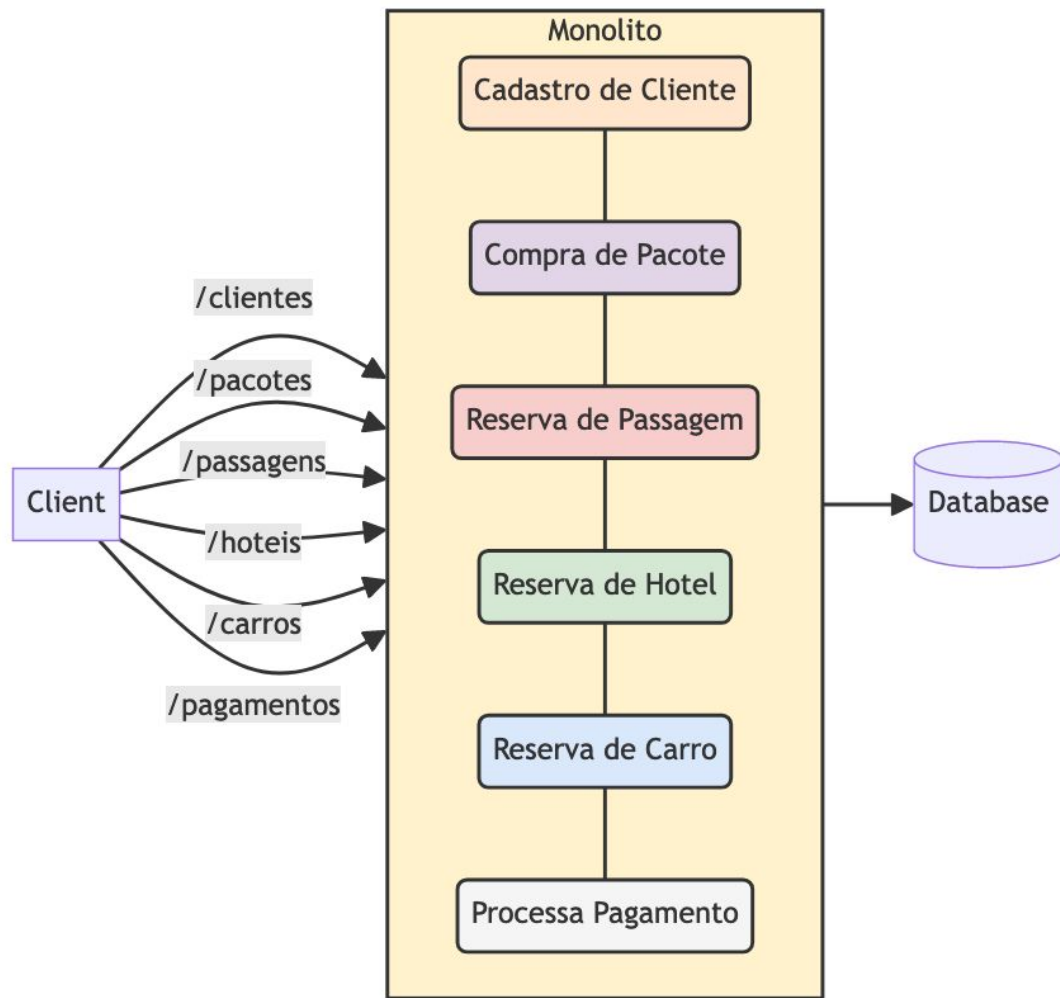
Arquitetura Monolitica

- Aplicação como unidade única e indivisível
- Todos os Componentes unificados
- Acesso local via métodos e componentes
- Mesmos:
 - Codebases
 - Binários
 - Aplicação
 - Dependências
 - Bancos de Dados
- MVP's*



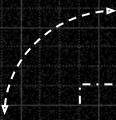
Arquitetura Monolitica - Vantagens

- Processos Unificados
- Gestão de Dependências
- Limita necessidades de protocolos externos (HTTP, gRPC, Mensageria)
- Facilidade de Consistencia de Dados
- Reduz Tempo e Energia para Validação
- Operação mais simples
- Facilidade de Testes de Integração



Arquitetura Monolitica – Desafios

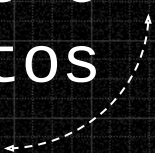
- Overhead
- Desafio de Escala
- Desafio de Manutenção
- Dificuldade de Escala Horizontal
- Dificuldade de Paralelização
- Necessidade constante de escala vertical
- Manutenção Degenerativa (times grandes)
- Desafio de Tolerância a Falhas
- Single Point of Failure
- Maior Blast Radius



1. Arquitetura de Microserviços



Definições e
Conceitos





Arquitetura de Microserviços

- Aplicação dividida em um conjunto de serviços menores e independentes
- Comunicação indireta via protocolos (HTTP, gRPC, mensageria)
- Cada microserviço focado em uma função de negócio específica.
- Desenvolvimento, implantação e escala independentes.

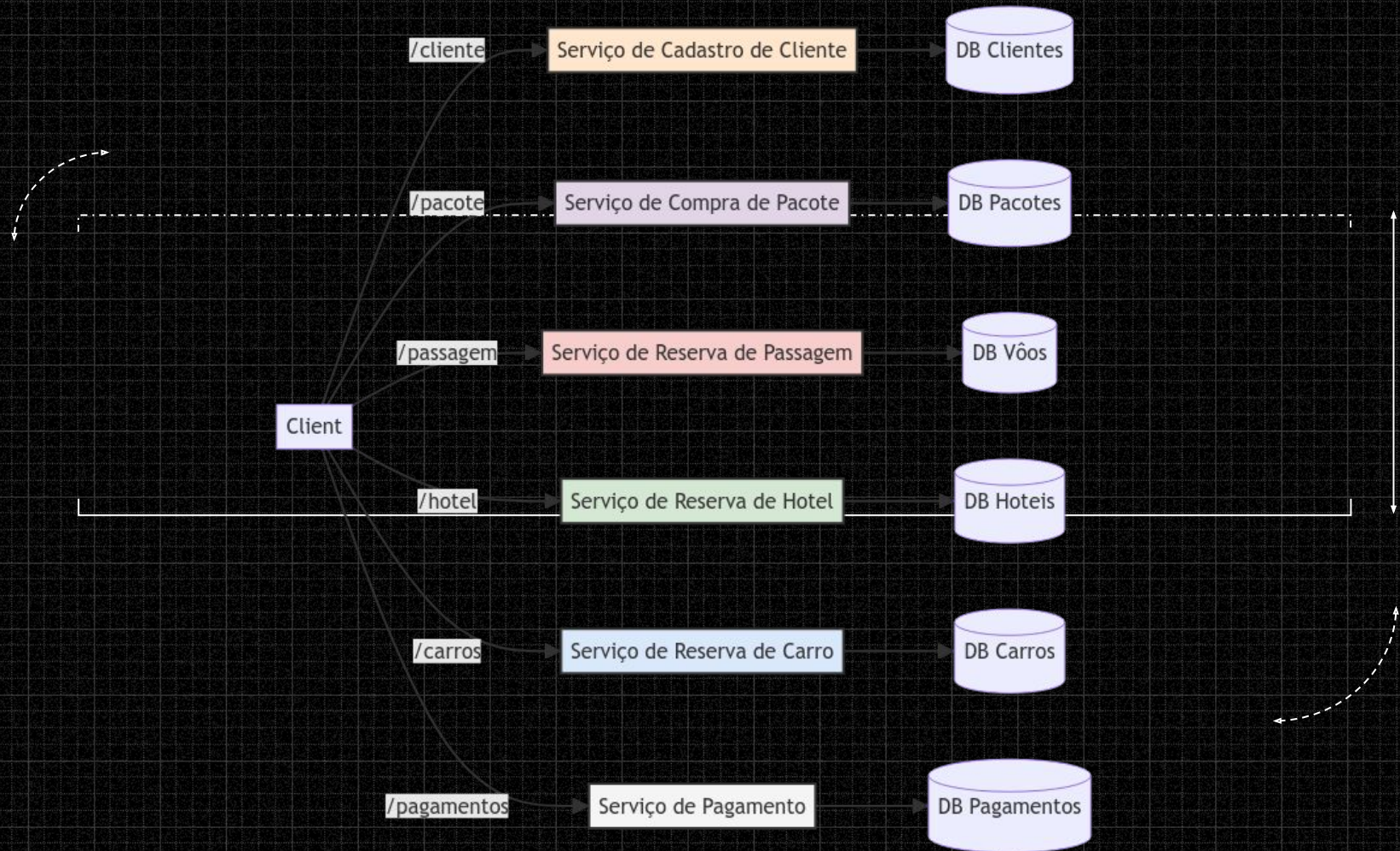


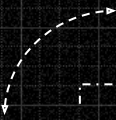
Arquitetura de Microserviços - Vantagens

- Descentralização
- Responsabilidade Reduzida
- Escala Independente
- Blast Radius Reduzido
- Autonomia dos Times
- Ciclo de Vida Isolado
- Melhor aplicabilidade de tecnologia

Arquitetura de Microserviços - Operações

- Complexidade Operacional
- Granularidade maior
- Comunicação por Rede (Latência, falhas de comunicação, necessidade de padrões de resiliência (circuit breakers, retries, DLQs)
- Testes Distribuídos
- Consistência de Dados Distribuídos
- Transações distribuídas
- Monitoramento e gestão de falhas.





1. Domain Driven Design

Definições e
Conceitos

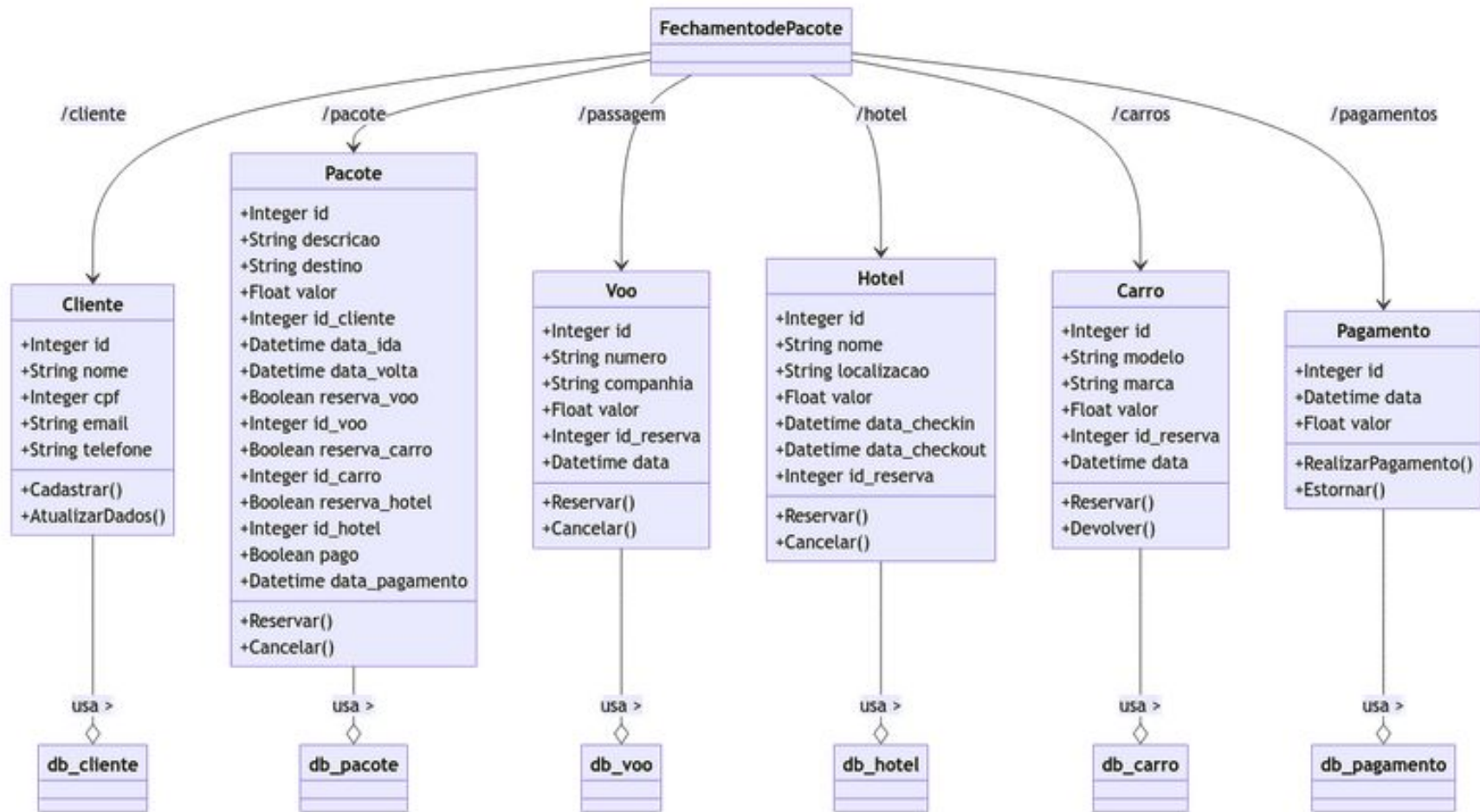


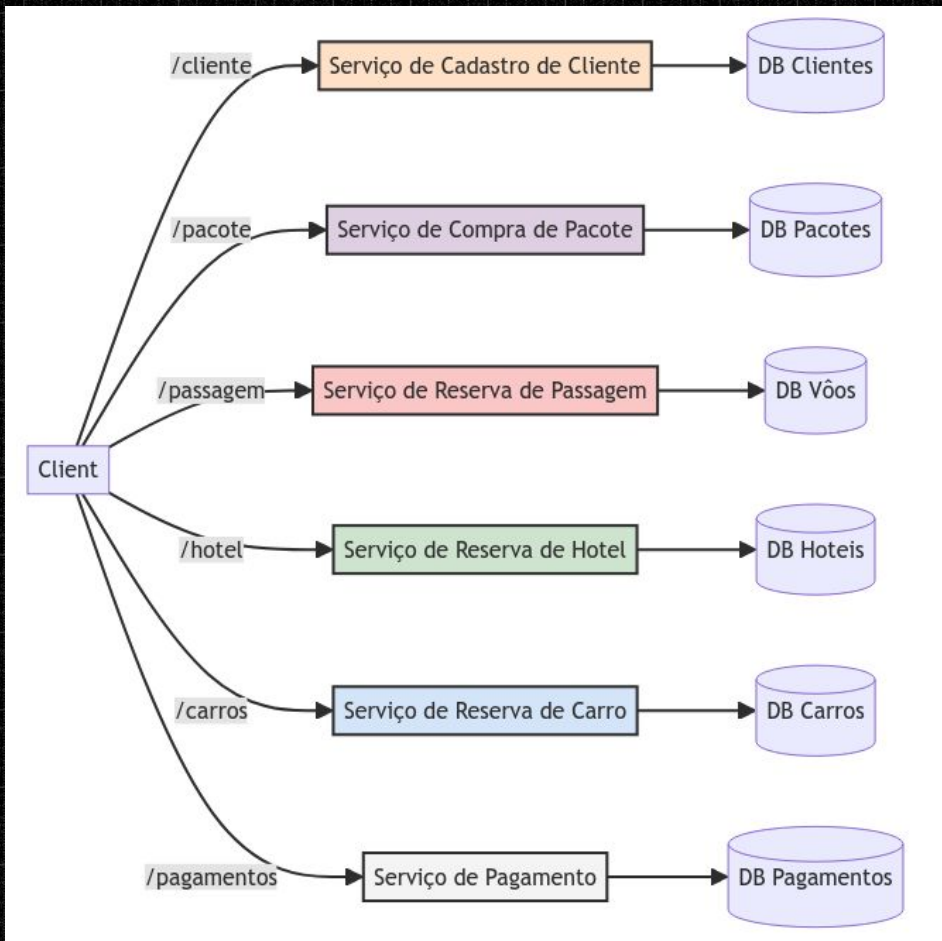
Domain Driven Design

- Foco no negócio
- Clareza e Alinhamento
- Abordagem que prioriza o domínio e a lógica de negócio no design
- Criação de limites claros e responsabilidades no sistema.
- Definição de Domínio de Negócio
- Modelagem Enriquecida: Entidades com comportamento e lógica de negócios
- Evitar 'modelos anêmicos'

Domain Driven Design

- Separação Clara de Domínios
- Identificar corretamente os limites de serviço
- Simplificando a gestão da lógica de domínio e reduzindo a complexidade operacional
- Não é exclusividade de Microserviços
- Definir limites de escopo entre classes e módulos dentro de uma aplicação monolítica
- Evitar a criação de sistemas diferentes, desenvolvidos por equipes distintas, que executam funções similares de forma redundante.





Microserviços não são
obrigatórios em DDD

Conceitos de DDD

- Domains
- Core-Domains
- Ubiquitous Language (Tecnologia e Negócios)
- Bounded Context
- Entities
- Value Objects
- Aggregates
- Domain Events
- Repository
- Services

Domains

- Área de conhecimento ou atividade da organização
 - Onde os problemas de negócio se manifestam
 - Base para criar o modelo de software
 - É o conjunto de regras, conceitos e processos que descrevem o negócio
-
- Exemplo: banco tem domínios como Crédito, Contas, Pagamentos, Empréstimos, Pix, Investimentos

Core Domains

- Parte mais importante e estratégica do negócio
- Diferencial Competitivo do Negócio
- Possui Domínios de Apoio
- S
- Uber -> Match de Motorista e Passageiro
- FinTechs -> Conta Digital e Cartão de Crédito
- iFood -> Fluxo de Pedidos

- ERP, RH, Backoffice, Analytics são domínios de suporte.

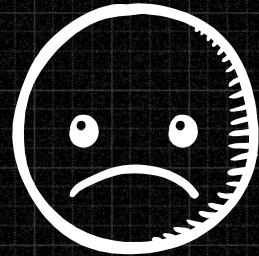
Linguagem Ubiqua

- Linguagem comum entre negócio e tecnologia
 - Evita ambiguidades e mal-entendidos
 - Termos devem ser absorvidos do domínio real
 - Time técnico deve combinar e adotar os termos do negócio
 - Evitar desvirtuar termos e criar "shadow language" no time técnico
-
- Usada em documentação, código e comunicação
 - Diagramas e ADR's

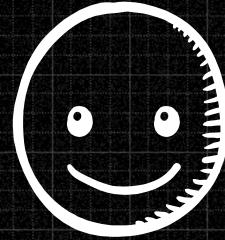
"Devemos criar um relatório de todos os clientes ativos no sistema"

Negócios: Clientes com atividade nos ultimos 90 dias

Tecnologia: Clientes Logados



"Devemos criar um relatório de todos os clientes ativos no sistema"

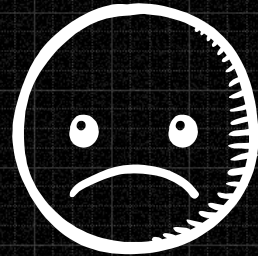


```
class Cliente {  
    boolean isAtivo() {  
        return dataUltimaMovimentacao.isAfter(hoje.minusDays(90));  
    }  
}
```

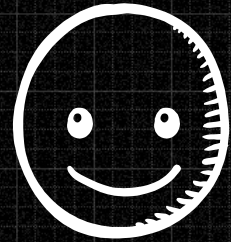
"Preciso notificar sempre que um pedido é confirmado"

Negócios: Um pedido confirmado é um pedido que o pagamento é aprovado pelo gateway de pagamento

Tecnologia: Pedidos registrados pelo cliente



"Um pedido CONFIRMADO é um pedido que foi aprovado pelo gateway de pagamento"



```
if (pagamento.status === "APROVADO") {  
    pedido.status = "CONFIRMADO";  
    evento.publish(new PedidoConfirmado(pedido.id));  
}
```

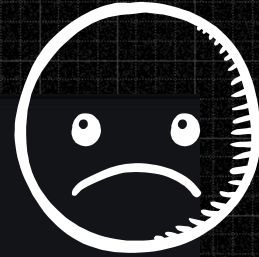
Entities

- Entidades Identificadas
- Objetos com identidade única e persistente
- Não é o valor que importa, mas o identificador
- Rastrear algo ao longo do tempo
- Exemplo: Cliente, Pedido, Conta Bancária
- Evitar Entidades Anêmicas

Entidades Anêmicas

- Entidades não devem ser apenas "sacos de dados"
- Encapsular também os comportamentos e métodos que opera sobre esses dados
- Refletir as regras de negócios com ações

Entidades Anêmicas



```
1  class ContaBancaria {
2      private String id;
3      private double saldo;
4
5      public String getId() { return id; }
6      public void setId(String id) { this.id = id; }
7      public double getSaldo() { return saldo; }
8      public void setSaldo(double saldo) { this.saldo = saldo; }
9  }
10
11 class ServicoBancario {
12     public void depositar(ContaBancaria conta, double valor) {
13         // Lógica de negócio aqui
14         conta.setSaldo(conta.getSaldo() + valor);
15     }
16 }
```


Entidades Ricas

```
1  class ContaBancaria {
2      private String id;
3      private double saldo;
4
5      public void depositar(double valor) {
6          // ...
7          if (valor <= 0) {
8              throw new IllegalArgumentException("...");
9          }
10         this.saldo += valor;
11         // ...
12     }
13
14     public void sacar(double valor) {
15         // ...
16         if (valor <= 0) {
17             throw new IllegalArgumentException("...");
18         }
19         if (this.saldo < valor) {
20             throw new IllegalStateException("Saldo insuficiente.");
21         }
22         this.saldo -= valor;
23         // ...
24     }
25 }
```

Value Objects

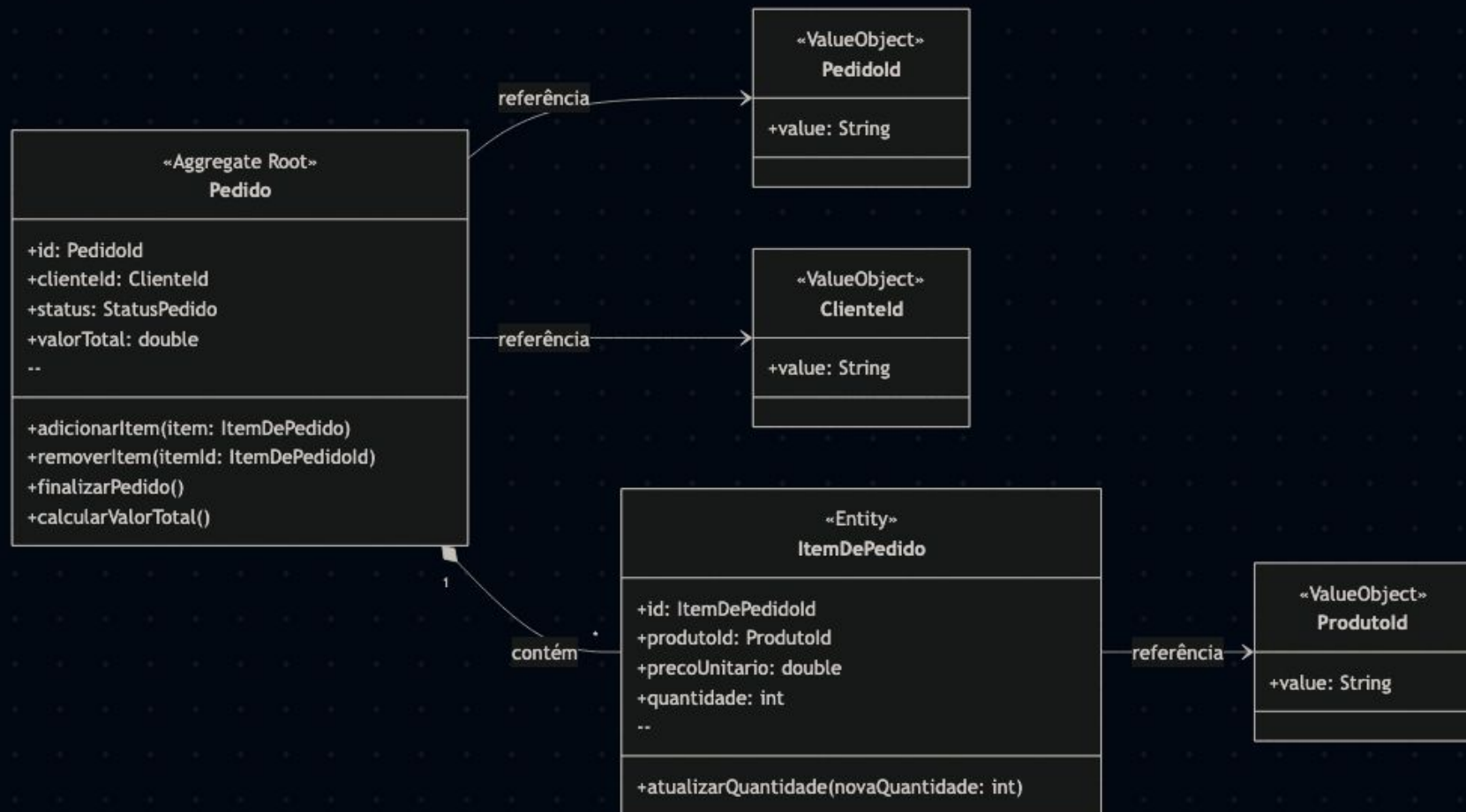
- Objetos de Valor
- Não possuem identidade
- Só possuem atributos
- São imutáveis (mudança = novo objeto)
- Exemplos: Endereço, Dinheiro, Coordenada GPS

Entities vs Value Objects

- Entidades possuem identificadores únicos
 - Objetos de Valor não possuem identidade
 - Entidades podem mudar atributos (menos id)
 - Objetos de valor geram um novo objeto
-
- Conta Bancária vs Saldo
 - CPF vs Endereço
 - ID vs Relacionamento

Aggregates

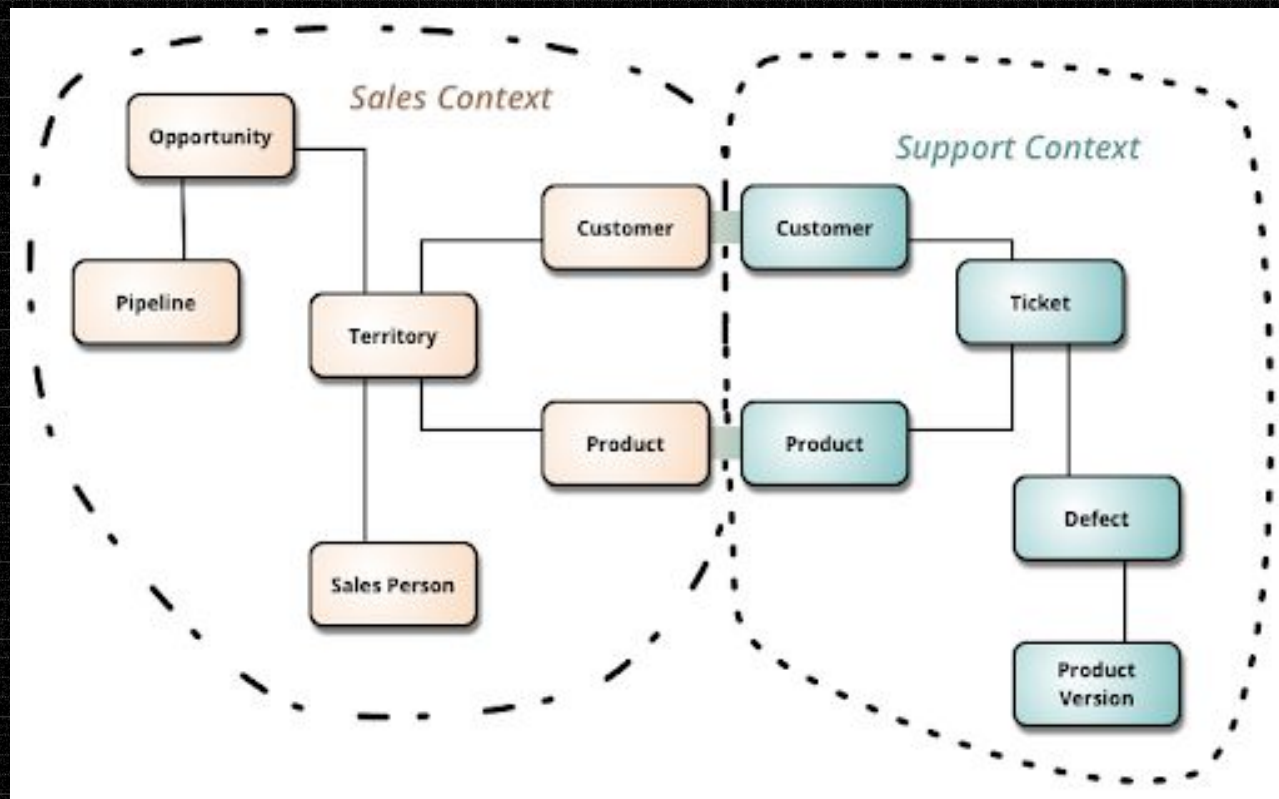
- Agregados
- Grupo coeso de Entidades e Objetos de Valor
- Conjunto de Entidades e Objetos de Domínio
- Possui uma Agregado Raiz que contém itens, valores e referências
- Precisam ser sempre consistentes uns com os outros
- Tratado como uma única unidade transacional



Bounded Context

- Fronteiras onde um modelo é válido
- Limites do Domínio Conhecido
- Evita que o mesmo termo tenha significados diferentes
- Cada contexto tem sua própria linguagem ubíqua
- Comunicação entre contextos via integrações
- Evita choque de significados.
- Isso naturalmente guia a divisão de módulos e microserviços.

Bounded Context

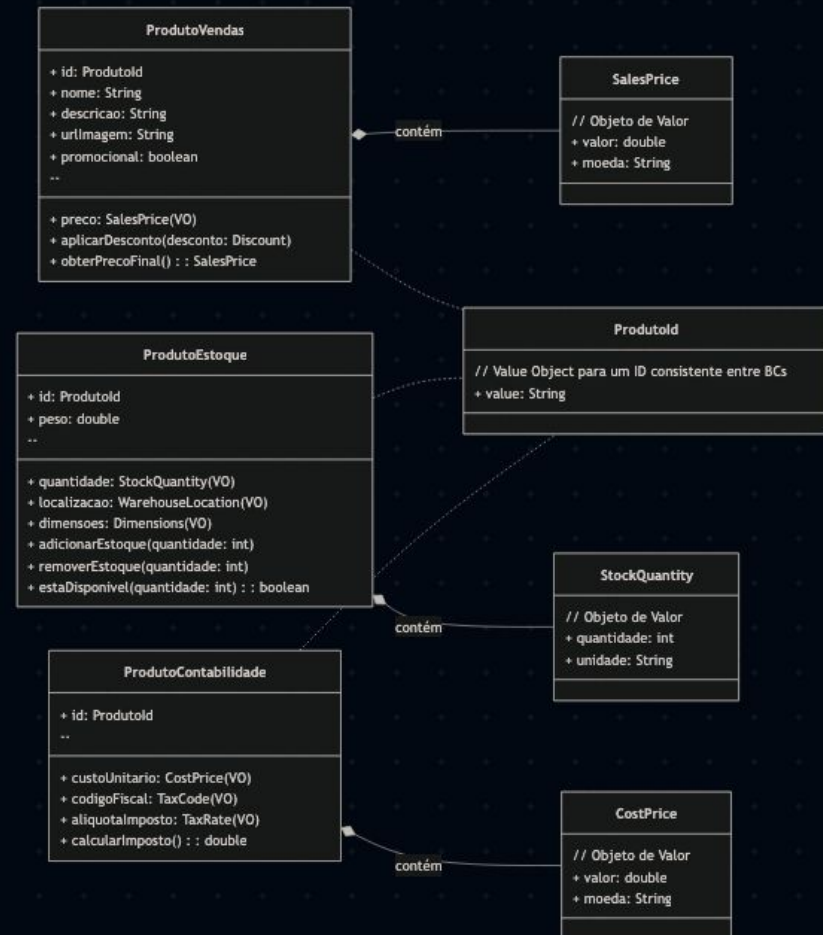


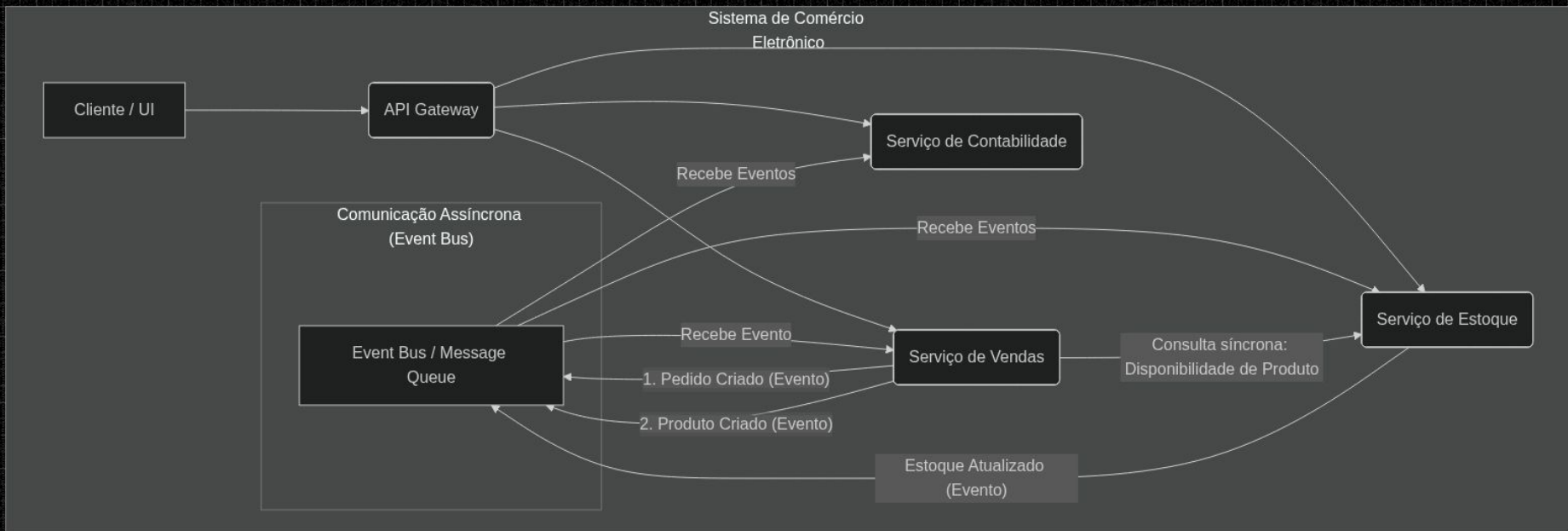
Bounded Context - God Object

- Mesmo Objeto "Produto"
 - Vendas
 - Estoque
 - Fiscal

Produto
+ id: String ---
+ nome: String(Vendas) + descricao: String(Vendas) + precoVenda: double(Vendas) + urlImagem: String(Vendas) + promocaoAtiva: boolean(Vendas) + quantidadeEstoque: int(Estoque) + localizacaoNoArmazem: String(Estoque) + pesoKg: double(Estoque) + dimensoesCm: String(Estoque) + dataUltimaReposicao: Date(Estoque) + custoUnitario: double(Contabilidade) + codigoFiscal: String(Contabilidade) + aliquotaImposto: double(Contabilidade) + calcularPrecoComDesconto() + atualizarEstoque() + gerarNotaFiscal() + validarDevolucao()

Bounded Context

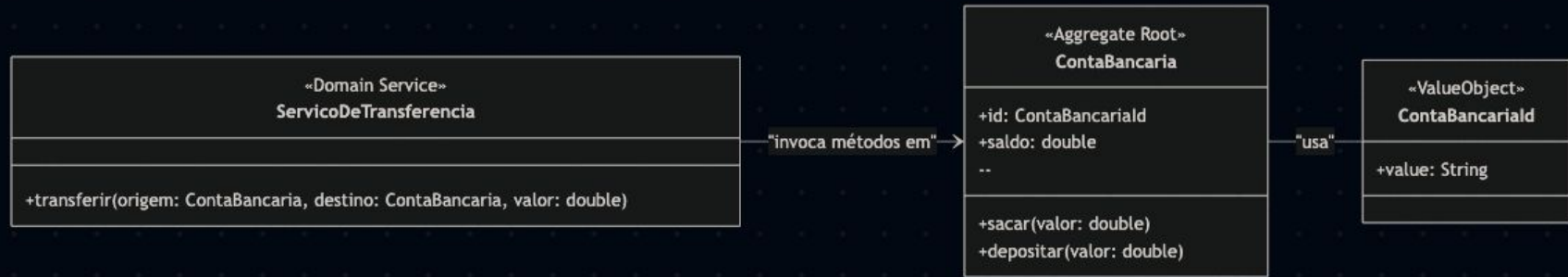


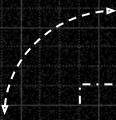


Services

- Serviços de Negócio
- Modelam operações importantes do negócio
- Usados quando lógica não cabe em uma única entidade
- Não possuem estado, apenas comportamento
- Exemplo: Serviço de Cálculo de Juros, Serviço de Conversão de Moeda, Transferência e etc

Services

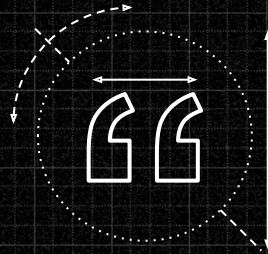




1. Lei de Conway na Arquitetura de Sistemas



Definições e
Conceitos



"A estrutura organizacional de uma empresa influencia diretamente a arquitetura do software que ela desenvolve."

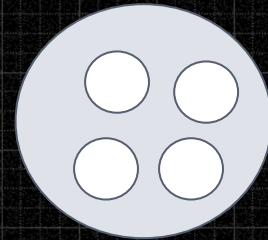
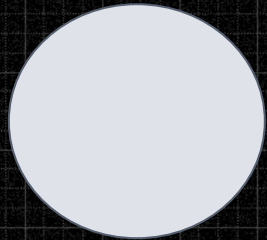
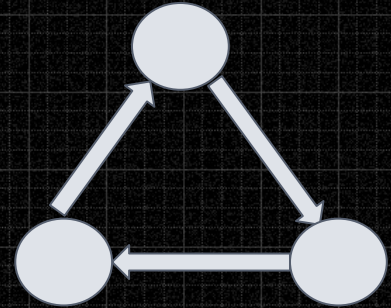
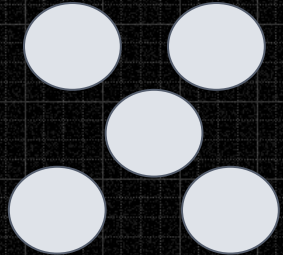
Lei de Conway - Curiosidades

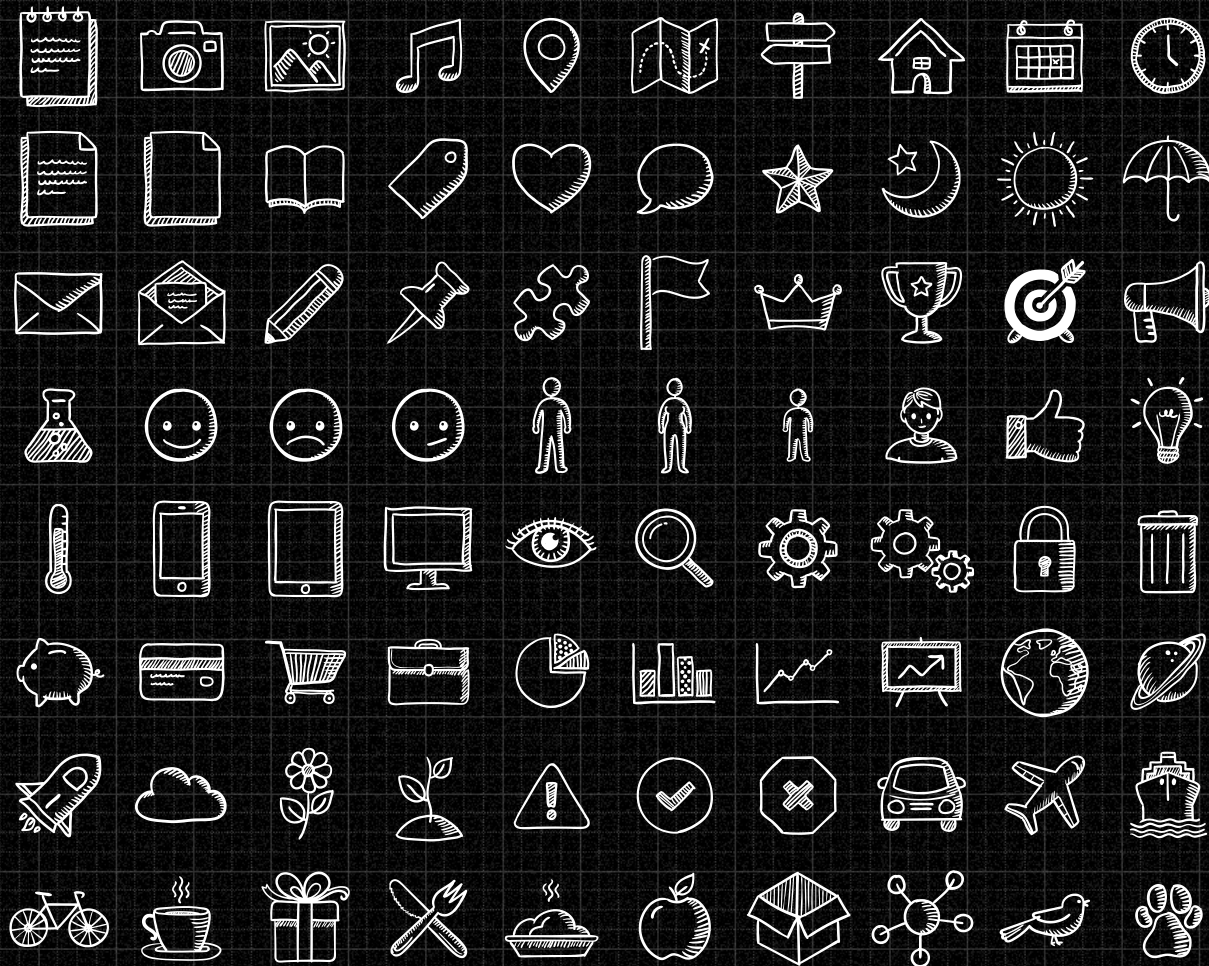
- Melvin Conway, 1967
- O paper foi rejeitado por Harvard
- "Não conseguiu provar a tese"
- Publicou a tese na revista "Datamanation"
- "Como os Comitês se Inventam*"
- Ganhou gigantesca notoriedade após a publicação

Lei de Conway

- Designs dos sistemas refletem a estrutura de comunicação das organizações.
- A forma como uma empresa está organizada
- Como as equipes são formadas
- Como as equipes se comunicam entre si
- Tende a moldar o desenvolvimento de Software

Lei de Conway





SlidesCarnival icons are editable shapes.

This means that you can:

- Resize them without losing quality.
- Change fill color and opacity.

Isn't that nice? :)

Examples:

